Codificación de variables categoricas

Tipo de Variable	Cardinalidad	Algoritmo	Técnica Recomendada
Ordinal (Hay orden)	Cualquier	Cualquiera	**Ordinal Encoding**
Nominal (Sin orden)	Baja ($< 10-15$ categorías)	Lineales / Redes / Árboles	**One-Hot Encoding**
Nominal (Sin orden)	Media / Alta	Árboles de Decisión / Ensamble	**Target / Frequency / Binary Encoding**
Nominal (Sin orden)	Muy Alta (> 1000)	Regresión / Redes	**Target Encoding / Hash Encoding**

Cuadro Comparativo de Opciones e Implementación

Método	Librería Recomendada	Opción Secundaria / Nativa	Casos de Uso Ideal
Ordinal Encoding	sklearn.preprocessing	Python Nativo (dict / map)	Variables con orden explícito (ej. Bajo, Medio, Alto).
One-Hot Encoding	sklearn.preprocessing	pandas.get_dummies()	Nominal con pocas categorías ($< 10-15$).
Target Encoding	category_encoders	sklearn.preprocessing	Alta cardinalidad (ej. Ciudades, Productos).
Frequency Encoding	pandas (Serie .map())	Python Nativo (collections.Counter)	Alta cardinalidad sin riesgo de fuga (*leakage*).
Binary Encoding	category_encoders	sklearn + operaciones bit	Cardinalidad media-alta ($10 - 100$ categorías).
Hash Encoding	sklearn.feature_extraction	category_encoders	Cardinalidad masiva o producción sin memoria de vocabulario.

1. Ordinal Encoding

Opción A: Scikit-Learn (OrdinalEncoder)

Python

```
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder

df = pd.DataFrame(
    {
        "Nivel_Educativo": [
            "Secundaria",
            "Licenciatura",
            "Maestria",
            "Secundaria",
        ]
    }
)

# Definimos explícitamente el orden deseado
categorias_ordenadas = [["Secundaria", "Licenciatura", "Maestria"]]

encoder = OrdinalEncoder(categories=categorias_ordenadas)
df["Educativo_Encoded"] = encoder.fit_transform(df[["Nivel_Educativo"]])

print(df)
```

Opción B: Python Nativo + Pandas (.map())

Python

```
import pandas as pd

df = pd.DataFrame(
    {
        "Nivel_Educativo": [
            "Secundaria",
            "Licenciatura",
            "Maestria",
            "Secundaria",
        ]
    }
)

# Mapeo manual usando diccionario nativo
orden_dict = {"Secundaria": 0, "Licenciatura": 1, "Maestria": 2}
df["Educativo_Encoded"] = df["Nivel_Educativo"].map(orden_dict)

print(df)
```

2. One-Hot Encoding

Opción A: Scikit-Learn (OneHotEncoder) — *Recomendado para Pipelines*

Python

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder

df = pd.DataFrame({"Combustible": ["Nafta", "Diesel", "GNC", "Nafta"]})

# sparse_output=False devuelve una matriz densa / NumPy array
encoder = OneHotEncoder(sparse_output=False, drop=None)
encoded_array = encoder.fit_transform(df[["Combustible"]])

# Convertir a DataFrame con nombres de columnas
df_encoded = pd.DataFrame(
    encoded_array, columns=encoder.get_feature_names_out(["Combustible"])
)
print(df_encoded)
```

Opción B: Pandas (pd.get_dummies()) — *Rápido para EDA*

Python

```
import pandas as pd

df = pd.DataFrame({"Combustible": ["Nafta", "Diesel", "GNC", "Nafta"]})

df_encoded = pd.get_dummies(
    df, columns=["Combustible"], prefix="Combustible", dtype=int
)
print(df_encoded)
```

3. Target Encoding (Mean Encoding)

Opción A: Librería category_encoders (TargetEncoder) — *La más completa*

Python

```
import category_encoders as ce
import pandas as pd

df = pd.DataFrame(
    {
        "Ciudad": [
            "Auckland",
            "Wellington",
            "Auckland",
            "Christchurch",
            "Wellington",
        ],
        "Ventas": [120, 80, 140, 90, 110],  # Variable Target
    }
)

# smoothing aplica un suavizado Bayesiano para prevenir overfitting
encoder = ce.TargetEncoder(cols=["Ciudad"], smoothing=10.0)
df["Ciudad_Encoded"] = encoder.fit_transform(df["Ciudad"], df["Ventas"])

print(df)
```

Opción B: Scikit-Learn (TargetEncoder)

Python

```
import pandas as pd
from sklearn.preprocessing import TargetEncoder

df = pd.DataFrame(
    {
        "Ciudad": [
            "Auckland",
            "Wellington",
            "Auckland",
            "Christchurch",
            "Wellington",
        ],
        "Ventas": [120, 80, 140, 90, 110],
    }
)

encoder = TargetEncoder(smooth="auto", cv=5)
# Requiere reshape o DataFrame para la target
df["Ciudad_Encoded"] = encoder.fit_transform(df[["Ciudad"]], df["Ventas"])

print(df)
```

4. Frequency / Count Encoding

Opción A: Pandas Nativo (.value_counts()) — *La forma estándar*

Python

```
import pandas as pd

df = pd.DataFrame(
    {
        "Marca": [
            "Toyota",
            "Ford",
            "Toyota",
            "Toyota",
            "Nissan",
            "Ford",
        ]
    }
)

# 1. Frecuencia absoluta (conteos)
frecuencias = df["Marca"].value_counts()
df["Marca_Count_Encoded"] = df["Marca"].map(frecuencias)

# 2. Frecuencia relativa (porcentaje / proporción)
frecuencias_rel = df["Marca"].value_counts(normalize=True)
df["Marca_Freq_Encoded"] = df["Marca"].map(frecuencias_rel)

print(df)
```

Opción B: Python Nativo (collections.Counter)

Python

```
from collections import Counter

marcas = ["Toyota", "Ford", "Toyota", "Toyota", "Nissan", "Ford"]

# Conteo nativo
conteo = Counter(marcas)
marcas_encoded = [conteo[m] for m in marcas]

print(marcas_encoded)  # [3, 2, 3, 3, 1, 2]
```

5. Binary Encoding

Opción A: Librería category_encoders (BinaryEncoder)

Python

```
import category_encoders as ce
import pandas as pd

df = pd.DataFrame(
    {
        "Provincia": [
            "Buenos Aires",
            "Cordoba",
            "Santa Fe",
            "Mendoza",
            "La Pampa",
            "Salta",
        ]
    }
)

encoder = ce.BinaryEncoder(cols=["Provincia"])
df_encoded = encoder.fit_transform(df)

print(df_encoded)
```

Opción B: sklearn + operaciones bit

Python

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder


def binary_encode_custom(df, column_name):
    # 1. Usar OrdinalEncoder para convertir categorías a enteros (0, 1, 2, ...)
    encoder = OrdinalEncoder()
    # Convertimos a entero de 64 bits para manipular bits
    ordinal_codes = (
        encoder.fit_transform(df[[column_name]]).astype(int).flatten()
    )

    # 2. Calcular cuántos bits (columnas) necesitamos
    # Se suma 1 al máximo código por si el índice arranca en 0
    max_code = ordinal_codes.max()
    n_bits = (
        int(np.ceil(np.log2(max_code + 1))) if max_code > 0 else 1
    )  # Al menos 1 bit

    # 3. Extraer cada bit usando Operaciones de Bits (Bitwise Operations)
    # Creamos una matriz para almacenar las columnas binarias
    binary_matrix = np.zeros((len(df), n_bits), dtype=int)

    for bit_idx in range(n_bits):
        # Desplazamiento a la derecha (>>) y máscara bitwise AND (& 1)
        # Extrae el bit en la posición (n_bits - 1 - bit_idx)
        shift_amount = n_bits - 1 - bit_idx
        binary_matrix[:, bit_idx] = (ordinal_codes >> shift_amount) & 1

    # 4. Crear un DataFrame con los nombres de columnas formateados
    bit_colnames = [f"{column_name}_bit_{i}" for i in range(n_bits)]
    df_binary = pd.DataFrame(
        binary_matrix, columns=bit_colnames, index=df.index
    )

    return pd.concat([df, df_binary], axis=1)
```

6. Hash Encoding (Hashing Trick)

Opción A: Scikit-Learn (FeatureHasher) — *Ultra eficiente en memoria*

Python

```
import pandas as pd
from sklearn.feature_extraction import FeatureHasher

df = pd.DataFrame({"Producto": ["Surtidor_A", "Surtidor_B", "Surtidor_A", "Filtro_C"]})

# n_features define el número fijo de columnas destino
hasher = FeatureHasher(n_features=4, input_type="string")

# FeatureHasher espera una lista de strings o iterables por fila
hashed_features = hasher.fit_transform([[x] for x in df["Producto"]])

# Convertir la matriz dispersa (sparse) a DataFrame
df_hashed = pd.DataFrame(hashed_features.toarray(), columns=[f"hash_{i}" for i in range(4)])
print(df_hashed)
```

Opción B: Librería category_encoders (HashingEncoder)

Python

```
import category_encoders as ce
import pandas as pd

df = pd.DataFrame({"Producto": ["Surtidor_A", "Surtidor_B", "Surtidor_A", "Filtro_C"]})

# Define 4 columnas hash directamente sobre el DataFrame
encoder = ce.HashingEncoder(cols=["Producto"], n_components=4)
df_encoded = encoder.fit_transform(df)

print(df_encoded)
```